# Phase 13: Request Schema

## Hospital Patient Helpdesk Chatbot

**Python module:** `07_backend/13_request_schema.py` **Jupyter notebook:** `13_notebooks/13_request_schema.ipynb` **Purpose:** Define and validate the user input format for the backend `/chat` endpoint.

## 1. Phase objective

Phase 13 creates the canonical request contract for the Hospital Patient Helpdesk Chatbot backend. It validates raw user input before the API passes a question into retrieval, prompting, generation, or guardrails.

The module is framework-independent. It works without FastAPI or Pydantic, but it can create a compatible Pydantic model when those dependencies are installed.

## 2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- sample_queries/
|   |   `-- test_questions.csv
|   `-- processed/
|       |-- 13_request_validation_results.json
|       |-- 13_request_schema_report.json
|       |-- 13_request_schema_audit.csv
|       |-- 13_failed_request_schema_checks.json
|       `-- plots/
|           |-- 13_question_length_distribution.png
|           `-- 13_request_validation_outcomes.png
|-- 07_backend/
|   |-- 13_request_schema.py
|   |-- 13_request_schema.md
|   `-- 13_request_schema.pdf
|-- 12_tests/
|   `-- test_request_schema.py
|-- 13_notebooks/
|   `-- 13_request_schema.ipynb
`-- README.md
```

All generated Phase 13 artifacts begin with `13_`.

## 3. Request fields

| Field | Required | Rule | Purpose |
|---|---|---|---|
| `question` | Yes | 2 to 1000 characters after whitespace normalization | Patient helpdesk question. |
| `department` | No | Up to 100 safe text characters | Optional retrieval filter. |
| `content_category` | No | Up to 100 safe text characters | Optional category filter. |
| `session_id` | No | Up to 64 letters, numbers, dot, underscore, colon, or hyphen | Optional client correlation ID. |
| `language` | No | Short code such as `en` or `en-US` | Future localization signal. |
| `channel` | No | Up to 32 letters, numbers, underscores, or hyphens | Caller source such as notebook, app, or api. |
| `urgency` | No | `routine`, `urgent`, or `emergency` | User-declared urgency signal. |

Unknown fields are ignored with a warning. This lets sample CSV columns such as `category`, `expected_source`, and `safety_class` remain in evaluation data without becoming API inputs.

# 4. Input file

## `01_data/sample_queries/test_questions.csv`

The batch validation uses the synthetic test questions as approved valid examples. The module also adds six adversarial request-shape cases:

- question too short;
- question too long;
- sensitive identifier in question;
- optional filter too long;
- invalid language code; and
- unknown field accepted with a warning.

# 5. Output files

| Numbered artifact | Description |
|---|---|
| `13_request_validation_results.json` | Full validation results, including normalized valid requests and errors for invalid requests. |
| `13_request_schema_report.json` | Totals, configuration, error counts, length statistics, dependency availability, and artifact inventory. |
| `13_request_schema_audit.csv` | Compact rows without full question text. |
| `13_failed_request_schema_checks.json` | Invalid request numbers and validation errors. |
| `13_question_length_distribution.png` | Histogram of request question lengths. |
| `13_request_validation_outcomes.png` | Valid versus invalid request counts. |

The plots are schema diagnostics. They do not measure answer quality or medical safety.

# 6. Python module code sections

## Constants and patterns

The module centralizes length limits, default values, allowed urgency values, safe-text pattern, session pattern, and sensitive-identifier patterns.

## Configuration and contracts

`RequestValidationConfig` controls minimum and maximum question length, optional-filter length, and whether sensitive identifiers may be allowed for internal review.

`ChatRequest` is the canonical normalized request passed to later API code.

`RequestValidationResult` records whether one payload is valid, the normalized request if valid, errors, warnings, character count, and ignored fields.

`RequestSchemaRunResult` records generated file paths and counts for batch execution.

## Normalization and sensitive checks

`normalize_whitespace()` collapses repeated whitespace. `sensitive_labels()` detects configured sensitive identifiers such as SSN, API-key-like strings, and medical-record-number patterns.

## Optional field validation

`optional_text()` normalizes bounded optional fields and rejects unsupported characters or excessive length.

### Main validation

`validate_request_payload()` applies the full request policy. It validates question length, safe characters, sensitive identifiers, optional filters, session ID, language code, channel, urgency, and unknown fields.

### Optional Pydantic model

`create_pydantic_request_model()` lazily imports Pydantic and returns a model compatible with FastAPI/OpenAPI. If Pydantic is unavailable, it raises a clear installation message while the core schema still works.

### Batch validation

`run_request_schema_validation()` loads sample questions, appends adversarial cases, validates all payloads, writes JSON and CSV artifacts, and creates the two plots.

### CLI

```
python 07_backend/13_request_schema.py
```

Optional flags:

```
python 07_backend/13_request_schema.py --sample-queries path/to/test_questions.csv --output-dir path/to/processed
```

## 7. Notebook code sections

The notebook:

1. locates the project from workspace, project root, or notebook folder;
2. imports the shared request-schema module;
3. displays Pydantic availability;
4. demonstrates a valid normalized request;
5. demonstrates invalid request cases;
6. checks the optional Pydantic model behavior;
7. runs the full 18-payload validation batch;
8. validates generated artifacts; and
9. displays both numbered plots.

## 8. Notebook versus Python module

| Topic | `13_request_schema.ipynb` | `13_request_schema.py` |
|---|---|---|
| Main purpose | Interactive explanation, examples, assertions, and plots. | Reusable canonical request validation. |
| Validation logic | Imports module functions. | Owns all rules, dataclasses, Pydantic factory, batch execution, artifacts, and CLI. |
| Inputs | Uses sample CSV and inline examples. | Accepts any compatible CSV through CLI. |
| Outputs | Displays examples, report, and plots. | Writes `13_` JSON, CSV, and PNG artifacts. |
| Dependency behavior | Reports whether Pydantic is installed. | Works without Pydantic; creates model only when installed. |

The notebook does not duplicate validation rules, so both files remain aligned.

## 9. Automated tests

`12_tests/test_request_schema.py` covers:

- valid request normalization;
- ignored unknown fields;
- too-short and too-long questions;
- sensitive identifier rejection;
- invalid language and urgency rejection;
- optional sensitive-identifier allowance for internal review; and
- optional Pydantic model creation when installed.

In the current interpreter, pytest and Pydantic are not installed, so direct module and notebook validation were used.

# 10. Validation results

The included run produced:

- 18 input payloads;
- 13 valid payloads;
- 5 invalid payloads;
- 12 approved sample questions accepted;
- 1 unknown-field adversarial payload accepted with a warning;
- 5 malformed or sensitive adversarial payloads rejected; and
- `pydantic_available: false` in the current environment.

# 11. Safety and privacy notes

- Do not put protected health information in test or demo requests.
- Rejecting obvious sensitive identifiers is not complete de-identification.
- Request validation does not replace Phase 11 output guardrails.
- Unknown fields are ignored rather than preserved to reduce accidental data retention.
- Audit output avoids storing full question text.
- Production APIs should add authentication, rate limiting, logging controls, retention limits, and privacy review.